SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANSWERS

Answer 1: HTTP Response Analysis

1. Incorrect or invalid fields.

Field	Status	Reason
Status line: HTTP/1.1 200 OK	Valid syntax	Correct format, but the status does not match the faulty body framing (see part 3).
Date: Mon, 01 Apr 2026 10:00:00 GMT	**Incorrect**	The format is valid, but 1 April 2026 is a **Wednesday**, not a Monday. The day name does not match the date.
Content-Type: text/html	Valid (incomplete)	Valid, but a charset is recommended: text/html; charset=UTF-8.
Content-Length: -150	**Invalid**	The value must be a non-negative integer giving the body size in bytes. A negative length is meaningless.
Connection: keep-alive	Valid	Allowed in HTTP/1.1 (persistent connections are the default).

2. Issue with Content-Length and its impact. Content-Length tells the client exactly how many bytes of body to read, and with persistent connections it defines where one response ends and the next begins. A value of -150 violates the grammar (1*DIGIT) in the HTTP specification. Possible impacts:

- The client cannot determine the body size, so many browsers and libraries reject the response as malformed (for example a header-parse or invalid-response error) or discard the body.
- Because the connection is *keep-alive*, the broken framing can desynchronize the connection. The client may hang waiting for data or misread the start of the next response.
- Proxies and caches may drop the response or mishandle it, and inconsistent length handling is a known cause of request/response smuggling vulnerabilities.
- The page may load partially or not at all, which harms user experience.

3. Does the status code match the response correctness? No. *200 OK* promises a successful, well-formed response, but the message itself is malformed because of the invalid Content-Length. A response with this fault does not correctly represent success. If the server cannot compute a valid length, it should report the failure instead, for example **500 Internal Server Error**. If the server is able to send the page, the header must be fixed so that 200 OK is truthful.

4. Suggested corrections.

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

- Set Content-Length to the real, non-negative byte count of the body (150 here, as an example). Alternatively, use *Transfer-Encoding: chunked* and omit Content-Length when the size is not known in advance.
- Correct the weekday so that the Date header matches the calendar date (Wed, 01 Apr 2026), and generate it from the server clock using the standard HTTP-date format.
- Add *charset=UTF-8* to the Content-Type header.
- Fix the application bug that produced the negative length and add tests for response headers.

Answer 2: DOM Structure Analysis

1. Missing or improperly closed elements.

- ** ** has no closing tag. It is closed only implicitly when the parser reaches .
- **<!DOCTYPE html>** is missing from the top of the document.
- The *lang* attribute on <html> and a <*meta charset*> declaration are missing.
- The text *HelloWorld* has no space between the words, which is probably a content mistake (Hello World).

2. How the browser interprets the structure. HTML5 parsers are error tolerant and follow fixed error-recovery rules. On reaching , the parser sees that a is still open and closes it automatically first. The resulting DOM is:

```
html
 +-- head
|    +-- title ("TestPage")
 +-- body
     +-- div
         +-- p ("HelloWorld")
```

So a regular HTML parser produces the intended tree. A strict XML/XHTML parser behaves differently: the document is not well formed, so it shows a parsing error and does not render the page.

3. Impact on rendering and layout.

- With no doctype, the browser enters **quirks mode**. The box model, default font sizes inside tables, and vertical sizing can differ from standards mode, so the layout may vary across browsers.
- The paragraph is still displayed with its default margins inside the div, so a simple page looks acceptable. In a larger page, however, an unclosed element can swallow following content into the wrong parent and break the layout and CSS selectors.
- Missing charset and lang can cause wrong characters and weaker accessibility and SEO.

4. Corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Test Page</title>
</head>
<body>
   <div>
      <p>Hello World</p>
   </div>
</body>
</html>
```

Answer 3: GET vs POST in Login System

1. Differences in data transmission.

Aspect	GET	POST
Where data is sent	In the URL query string (after ?)	In the HTTP request body
Visibility	Visible in address bar, history, logs, Referer header	Not shown in the URL (but not encrypted without HTTPS)
Size limit	Limited by URL length (about 2 KB in practice)	No practical limit set by the protocol
Caching and bookmarking	Can be cached and bookmarked	Not cached or bookmarked by default
Semantics	Safe and idempotent, used to read data	Not idempotent, used to create or submit data
Example	GET /login?user=admin&pass=1234	POST /login with body user=admin&pass=1234

2. More suitable method for login. POST is the suitable method. A login request submits sensitive credentials and results in a server-side action (creating a session), so it should not appear in the URL or be cached, bookmarked or logged. POST keeps the credentials in the body, out of history and access logs, and matches the intended meaning of the method. POST alone is not secure, so it must be used together with HTTPS.

3. Security risks of using GET for authentication.

- **Credential exposure**: the password is displayed in the address bar and can be seen by shoulder surfing.
- **Browser history and bookmarks** store the full URL, including the password.
- **Server, proxy and CDN logs** record complete URLs, leaking credentials to anyone with log access.
- **Referer header leakage**: when the user clicks an external link, the URL may be sent to a third-party site.
- **Caching and sharing**: the URL can be cached, copied into messages, or replayed by anyone who obtains it.
- **CSRF and replay**: a login via GET can be triggered by an image tag or link on another site.

4. Best practices for secure login implementation.

1. Use POST over **HTTPS only**, with HSTS enabled.
2. Hash passwords on the server with a slow salted algorithm (bcrypt, scrypt, Argon2). Never store plain text.
3. Use CSRF tokens, and set session cookies with Secure, HttpOnly and SameSite flags. Regenerate the session ID after login.
4. Validate and sanitize input, and use parameterized queries to prevent SQL injection.
5. Apply rate limiting, account lockout or CAPTCHA against brute-force attacks.
6. Show generic error messages and never reveal whether the username or the password was wrong.
7. Add multi-factor authentication and avoid putting secrets in URLs, logs or client-side code.

Answer 4: HTML Code Inspection

1. Missing closing tags.

- **<h1>Welcome** has no </h1>.
- ** This is a paragraph** has no .
- The <img> element is a void element and does not need a closing tag in HTML5. In XHTML it must be written as .

2. Issues with the <img> tag.

- The **alt attribute is missing**. It is required by the HTML standard and is needed for screen readers, for SEO, and to show text if the image fails to load.
- **width and height** are not given, so the browser cannot reserve space and the page layout shifts when the image loads (poor Cumulative Layout Shift).
- The tag is not self-closed (/>), which is acceptable in HTML5 but invalid if the document is parsed as XHTML.
- The image sits inside the unclosed paragraph and heading context, so its placement is unclear.

3. How browsers handle these errors. Browsers do not stop on these errors. The HTML5 parsing algorithm repairs them. Because a <p> start tag does not close an open <h1>, the paragraph and the image end up nested inside the heading, so the text *This is a paragraph* is displayed in large bold heading style. All still-open elements are closed automatically when </body> is reached. The missing alt attribute causes no visual error, but it breaks accessibility and fails validation. Different engines generally produce the same tree in HTML5 mode, but a missing closing tag still makes the result unpredictable and hard to style.

4. Corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Sample</title>
</head>
<body>
   <h1>Welcome</h1>
   <p>This is a paragraph</p>
   <img src="image.jpg" alt="Descriptive text for the image"
      width="400" height="300">
</body>
</html>
```

Answer 5: Browser-Server Communication Flow

1. Interpretation of each step.

Step	What happens
1. User enters a URL	The browser parses the URL into scheme (https), host (www.example.com), port, path and query. It checks its cache, HSTS list and history before making a network request.

Step	What happens
2. DNS lookup	The domain name is translated into an IP address. The browser cache, the OS cache, the router, and then the ISP's recursive resolver are asked in turn, which may query root, TLD and authoritative name servers.
3. HTTP request is sent	The browser opens a TCP connection (three-way handshake) and, for HTTPS, completes the TLS handshake. It then sends a request such as GET / HTTP/1.1 with Host, User-Agent, Accept and Cookie headers.
4. Server processes request	The web server routes the request, runs application logic, queries databases or caches, and builds the content.
5. Response is returned	The server sends a status line (for example 200 OK), headers (Content-Type, Content-Length, Cache-Control) and the body, usually compressed.
6. Browser renders page	The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree, runs layout, paints pixels, and fetches extra resources (CSS, JS, images) which repeat steps 3 to 5.

2. Where delays can occur.

- **DNS lookup**: slow or distant resolvers, or a cache miss that requires several recursive queries.
- **TCP and TLS handshakes**: extra round trips, especially on high-latency or mobile networks.
- **Network latency and bandwidth**: physical distance to the server and congestion.
- **Server processing**: slow database queries, heavy computation, or an overloaded server (high Time To First Byte).
- **Large responses**: uncompressed or unoptimized HTML, images, scripts and fonts.
- **Redirect chains** and many separate requests for resources.
- **Rendering**: render-blocking CSS and JavaScript, and heavy DOM or layout work.

3. Role of DNS. People use human-readable names, but networks route traffic by IP address. DNS is the distributed, hierarchical directory that maps a domain name to an IP address (A or AAAA record). Without it, the browser has no destination for the TCP connection, so no request can be sent. DNS also supports load balancing and failover (multiple records or geo-based answers), and its results are cached based on the TTL value, which reduces repeated lookups. Because the request cannot start until DNS answers, a slow DNS directly adds to page load time.

4. Optimizations to improve performance.

- **DNS**: use a fast DNS provider, set sensible TTLs, and use *dns-prefetch* and *preconnect* hints.
- **Network**: use a CDN to serve content close to users, enable HTTP/2 or HTTP/3, TLS 1.3 and keep-alive connections.
- **Payload**: enable gzip or Brotli compression, minify HTML, CSS and JS, and use modern image formats (WebP, AVIF) with lazy loading.
- **Caching**: set Cache-Control and ETag headers for browser and CDN caching, and use server-side caching (for example Redis).
- **Server**: optimize database queries and indexes, and scale the back end horizontally.
- **Rendering**: load scripts with *async* or *defer*, inline critical CSS, and reduce redirects and the number of requests.